

Code ▾

Homework 3

Jonathan Dale

2026-04-24

Instructions: Include all codes and R-outputs that are necessary for each question. Submission: You MUST submit a PDF file.

Question 1: Regression Models:

Is there a relationship between student math test scores and socioeconomic variables? The data set `CASchools.csv` contains data on test performance, school demographics, and student demographic background for school districts in California. Remove the variable `county` before the analysis.

Please use the Data set description document to learn more about the data set.

a). Fit a linear regression model to predict math test scores using the variables in the dataset. Discuss your results, making sure to cover the following points:

1. What do you observe about the relationship between these predictors and math test scores?
2. Are there any insignificant variables?
3. Which predictor is explaining math scores the best?
4. Explain the interpretation of the coefficient on lunch in terms of the response variable.

Hide

```
library(tidyverse)
library(dplyr)

data_1a = read.csv("CASchools.csv")
# data_1a = data
glimpse(data_1a)
```

```
## Rows: 429
## Columns: 13
## $ district    <int> 75119, 61499, 61549, 61457, 61523, 62042, 68536, 63834, 62...
## $ school      <chr> "Sunol Glen Unified", "Manzanita Elementary", "Thermalito ...
## $ county      <chr> "Alameda", "Butte", "Butte", "Butte", "Butte", "Fresno", "...
## $ grades      <chr> "KK-08", "KK-08", "KK-08", "KK-08", "KK-08", "KK-08", "KK-...
## $ students    <int> 195, 240, 1550, 243, 1335, 137, 195, 888, 379, 2247, 446, ...
## $ teachers    <dbl> 10.90, 11.15, 82.90, 14.00, 71.50, 6.40, 10.00, 42.50, 19...
## $ calworks    <dbl> 0.5102, 15.4167, 55.0323, 36.4754, 33.1086, 12.3188, 12.90...
## $ lunch       <dbl> 2.0408, 47.9167, 76.3226, 77.0492, 78.4270, 86.9565, 94.62...
## $ computer    <chr> "67", "101", "169", "85", "171", "25", "28", "66", "35", "...
## $ expenditure <dbl> 6384.911, 5099.381, 5501.955, 7101.831, 5235.988, 5580.147...
## $ income      <chr> "22.69000053", "9.824000359", "8.977999687", "8.977999687"...
## $ english     <dbl> 0.000000, 4.583333, 30.000002, 0.000000, 13.857677, 12.408...
## $ math        <dbl> 690.0, 661.9, 650.9, 643.5, 639.9, 605.4, 609.0, 612.5, 61...
```

Hide

```
# Data analysis and cleaning:
# Remove the "county" column
# The "district" and "school" are ID attributes which are not useful for regression analysis,
# they are removed from the data and will not be use in the model.
# Two columns are character type, "computer" & "income", convert to numeric.
# The "grades" column is also character type, convert to factor for analysis.
data_1a = read.csv("CASchools.csv") |>
  select(-county, -district, -school) |>
  mutate(
    across(
      c(students, computer, income),
      ~ suppressWarnings(as.numeric(.x))
    ),
    grades = factor(grades)
  ) |>
  drop_na() # complete cases only

# Train / test split (80/20) for model evaluation.
set.seed(335)
n_1a = nrow(data_1a)
train_idx = sample.int(n_1a, size = floor(0.8 * n_1a))
train = data_1a[train_idx, , drop = FALSE]
test = data_1a[-train_idx, , drop = FALSE]

qlimpse(data_1a)
```

```
## Rows: 423
## Columns: 10
## $ grades      <fct> KK-08, KK-08, KK-08, KK-08, KK-08, KK-08, KK-08, KK-08, KK...
## $ students    <dbl> 195, 240, 1550, 243, 1335, 137, 195, 888, 379, 2247, 446, ...
## $ teachers     <dbl> 10.90, 11.15, 82.90, 14.00, 71.50, 6.40, 10.00, 42.50, 19....
## $ calworks     <dbl> 0.5102, 15.4167, 55.0323, 36.4754, 33.1086, 12.3188, 12.90...
## $ lunch        <dbl> 2.0408, 47.9167, 76.3226, 77.0492, 78.4270, 86.9565, 94.62...
## $ computer     <dbl> 67, 101, 169, 85, 171, 25, 28, 66, 35, 0, 86, 56, 25, 0, 3...
## $ expenditure  <dbl> 6384.911, 5099.381, 5501.955, 7101.831, 5235.988, 5580.147...
## $ income       <dbl> 22.690001, 9.824000, 8.978000, 8.978000, 9.080333, 10.4150...
## $ english      <dbl> 0.000000, 4.583333, 30.000002, 0.000000, 13.857677, 12.408...
## $ math         <dbl> 690.0, 661.9, 650.9, 643.5, 639.9, 605.4, 609.0, 612.5, 61...
```

Hide

```
# Multiple linear regression (ISLR2 §3.6.3)
# Source: ISLRv2 https://hastie.su.domains/ISLR2/ISLRv2\_corrected\_June\_2023.pdf.download.html
fit1 = lm(math ~ ., data = data1a)
```

```
## To see a list of hidden values use 'str(summary(model))'
## https://stackoverflow.com/a/5588638
# Extract just the p-values from the model (excluding the intercept)
# Filter for Insignificant variables where  $p > 0.05$ 
p_values = summary(fit1)$coefficients[, "Pr(>|t|)"][-1]
insignificant_vars = p_values[p_values > 0.05]
print(insignificant_vars)
```

```
##      students      teachers      calworks      computer expenditure
## 0.71871421 0.97624848 0.13047950 0.08422283 0.45933264
```

Hide

```
# Find the strongest predictor, scaling of all the variables
# Then create a new model using scaled variables.
data_1a_scaled = data_1a |>
  mutate(across(where(is.numeric), scale))

# Fit a new model with scaled values
fit1_scaled = lm(math ~ ., data_1a_scaled)

# Extract coefficients (excluding the intercept) to find the strongest predictor
betas = coef(fit1_scaled)[-1]
strongest_beta_name = names(which.max(abs(betas)))
strongest_beta_value = betas[strongest_beta_name]

c("Strongest Predictor:", strongest_beta_name)
```

```
## [1] "Strongest Predictor:" "lunch"
```

Hide

```
# Remove the insignificant variables to find largest |t|
data_1a_significant = data_1a_scaled |> select(-names(insignificant_vars))
names(data_1a_significant)
```

```
## [1] "grades" "lunch" "income" "english" "math"
```

Hide

```
fit1_sig = lm(math ~ ., data_1a_significant)

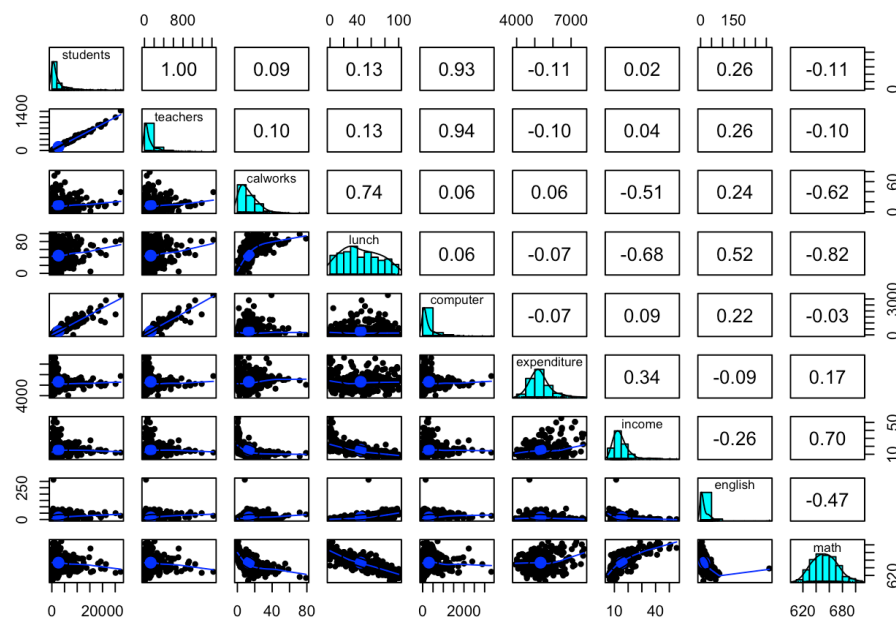
t_values_std = summary(fit1_sig)$coefficients[, "t value"][-1]
max_t_name = names(which.max(abs(t_values_std)))
max_t_value = t_values_std[max_t_name]

c("Max |t| value:", round(max_t_value, 4))
```

```
##                lunch
## "Max |t| value:" "-14.3829"
```

Hide

```
# # Create a pairwise correlation plot with the pairs function using only numeric columns
# # scatter plot matrix to observe the behavior of each variable
library(psych)
pairs.panels(data_1a |> select(where(is.numeric)))
```



Hide

```
# Simple regression models for comparison:
# math ~ lunch
fit_lunch = lm(math ~ lunch, data = data_1a)
math_vs_lunch = c(r_squared = summary(fit_lunch)$r.squared, residual_se = summary(fit_lunch)$sigma)

# math ~ income
fit_income = lm(math ~ income, data = data_1a)
math_vs_income = c(r_squared = summary(fit_income)$r.squared, residual_se = summary(fit_income)$sigma)

# Extract R-squared and residual SE (sigma) from each fit for side-by-side comparison
original_model = c(
  r_squared = summary(fit1)$r.squared,
  residual_se = summary(fit1)$sigma
)

comparison_1a = tibble(
  model = c(
    "Full: math ~ . (all predictors)",
    "Simple: math ~ lunch",
    "Simple: math ~ income"
  ),
  r_squared = c(
    unname(original_model["r_squared"]),
    unname(math_vs_lunch["r_squared"]),
    unname(math_vs_income["r_squared"])
  ),
  residual_se = c(
    unname(original_model["residual_se"]),
    unname(math_vs_lunch["residual_se"]),
    unname(math_vs_income["residual_se"])
  )
)

print(comparison_1a)
```

```
## # A tibble: 3 × 3
##   model                                r_squared residual_se
##   <chr>                                <dbl>         <dbl>
## 1 Full: math ~ . (all predictors)      0.731          9.89
## 2 Simple: math ~ lunch                 0.679         10.7
## 3 Simple: math ~ income                 0.495         13.4
```

Hide

```
# knitr::kable(
#   comparison_1a,
#   digits = 4,
#   col.names = c("Model", "R-squared", "Residual standard error"),
#   caption = "Full multiple regression versus one-predictor models (same training data)."
# )
```

Discussion (Question 1a).

- Predictors vs math.** The full regression model shows strong negative association with `lunch` (Percent qualifying for reduced-price lunch), and a smaller positive association with `income` and `math` both clearly displayed in the plot matrix. The `expenditure`, `students`, `teachers`, and `calworks` are weaker in this fit after adjusting for the others.
 - Insignificant values ($p > 0.05$).** I used a filter to sort parameters where $p > 0.05$, this means the parameter is insignificant and will not contribute anything meaningful to the model. Store these values inside a variable named `insignificant_vars`. The following variables have p-values above 0.05: `students`, `teachers`, `calworks`, `expenditure`; also `computer` is marginal though still above the threshold.
 - Strongest predictor.** By evidence in the data, it is clear that `lunch` has the largest $|t|$ and smallest p-value. among the predictors making it the strongest linear signal in this model. This is found by scaling all the parameters in the data set.
 - `lunch` coefficient.** With other predictors held fixed, each **one percentage-point** increase in `lunch` is associated with an estimated **decrease** of about **0.37** points in average `math` test score. In other words, areas with higher percent of students qualifying for reduced-price lunch, our model predicts a significantly larger portion of students with lower average math scores than areas with lower percentage of students on reduced-prices lunch.
- b). Do you think you can fit a better model than the model in part (a)? Fit a new model that you think is a better fit for the data. What do you observe about this new model? (Use an appropriate variable selection algorithm and use cross-validation to validate your answers).

Hide

```
library(caret)
set.seed(335)

# Start from the full model and use bidirectional stepwise selection.
model_full = lm(math ~ ., data = data_1a)
model_step = stepAIC(model_full, direction = "both", trace = 0)
summary(model_step)
```

```
##
## Call:
## lm(formula = math ~ grades + students + calworks + lunch + computer +
##     income + english, data = data_1a)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -29.8454  -6.7140  -0.1063   5.7955  31.3953
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.651e+02  2.670e+00  249.145 < 2e-16 ***
## gradesKK-08 -3.836e+00  1.387e+00  -2.766  0.00593 **
## students    -7.012e-04  3.426e-04  -2.047  0.04132 *
## calworks    -9.258e-02  6.504e-02  -1.423  0.15536
## lunch       -3.645e-01  3.683e-02  -9.897 < 2e-16 ***
## computer     5.732e-03  3.025e-03   1.895  0.05883 .
## income       6.869e-01  9.151e-02   7.506  3.75e-13 ***
## english     -8.479e-02  2.616e-02  -3.241  0.00129 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.869 on 415 degrees of freedom
## Multiple R-squared:  0.7302, Adjusted R-squared:  0.7256
## F-statistic: 160.4 on 7 and 415 DF, p-value: < 2.2e-16
```

Hide

```

# Use the selected predictors to define a reduced formula.
selected_terms = attr(terms(model_step), "term.labels")
formula_step = as.formula(paste("math ~", paste(selected_terms, collapse = " + ")))

# Compare full vs selected model with the same CV scheme.
ctrl = trainControl(method = "repeatedcv", number = 10, repeats = 5)
cv_full = train(
  math ~ .,
  data = data_1a,
  method = "lm",
  trControl = ctrl,
  na.action = na.omit
)
cv_step = train(
  formula_step,
  data = data_1a,
  method = "lm",
  trControl = ctrl,
  na.action = na.omit
)

# Model-comparison table.
comparison_1b = tibble(
  model = c(
    "Full model (part a)",
    "Stepwise-selected model (part b)"
  ),
  cv_rmse = c(
    cv_full$results$RMSE[1],
    cv_step$results$RMSE[1]
  ),
  cv_mse = cv_rmse^2,
  cv_r2 = c(
    cv_full$results$Rsquared[1],
    cv_step$results$Rsquared[1]
  ),
  adj_r2 = c(
    summary(model_full)$adj.r.squared,
    summary(model_step)$adj.r.squared
  ),
  aic = c(
    AIC(model_full),
    AIC(model_step)
  ),
  bic = c(
    BIC(model_full),
    BIC(model_step)
  )
)

anova_compare = anova(model_full, model_step)

cat("Selected predictors in stepwise model:\n")

```

```
## Selected predictors in stepwise model:
```

Hide

```
print(selected_terms)
```

```
## [1] "grades" "students" "calworks" "lunch" "computer" "income" "english"
```

Hide

```
cat("\nCross-validated comparison:\n")
```

```
##
## Cross-validated comparison:
```

Hide

```
print(comparison_1b)
```

```
## # A tibble: 2 × 7
##   model                cv_rmse cv_mse cv_r2 adj_r2   aic   bic
##   <chr>                <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Full model (part a)    10.1   102. 0.717 0.725 3151. 3195.
## 2 Stepwise-selected model (part b) 10.0   100. 0.720 0.726 3147. 3184.
```

Hide

```
cat("\nNested model ANOVA (full vs stepwise):\n")
```

```
##
## Nested model ANOVA (full vs stepwise):
```

Hide

```
print(anova_compare)
```

```
## Analysis of Variance Table
##
## Model 1: math ~ grades + students + teachers + calworks + lunch + computer +
##   expenditure + income + english
## Model 2: math ~ grades + students + calworks + lunch + computer + income +
##   english
##   Res.Df    RSS Df Sum of Sq    F Pr(>F)
## 1      413 40364
## 2      415 40417 -2    -53.933 0.2759 0.759
```

Hide

```
# source: https://topepo.github.io/caret/subsampling-for-class-imbalances.html
```

Discussion (Question 1b).

1.b **Model comparison.** I used bidirectional stepwise selection (AIC criterion) to obtain a reduced model and then validated both the original full model and the reduced model using repeated 10-fold cross-validation. The selected model keeps only predictors that improve fit after accounting for the others, making it more interpretable with fewer terms.

Comparing cross-validated MSE (and CV R^2) between models is the key evidence for whether part (b) is truly better than part (a). If the stepwise model has lower CV MSE and similar or higher CV R^2 , then it is a better out-of-sample model; if metrics are very close, the reduced model is still preferable for parsimony.

c). Write the equation for the least square fit from part b).

Hide

```
# Display the coefficients
round(coef(model_step), 2)
```

```
## (Intercept) gradesKK-08    students    calworks    lunch    computer
##      665.10      -3.84         0.00      -0.09     -0.36         0.01
##      income      english
##       0.69       -0.08
```

Discussion (Question 1c).

1.c **Least squares equation.** Following the formula: $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$ and using the coefficients function we can see the exact values for each beta value. Combining these two pieces to form the equation the least square fit:

$Y = 665.10 - 3.84 * \text{gradesKK-08} + 0.00 * \text{students} - 0.09 * \text{calworks} - 0.36 * \text{lunch} + 0.01 * \text{computer} + 0.69 * \text{income} - 0.08 \text{ en}$

d). Compare the R2 and Testing MSE for models from part (a) and (b).

Hide

```
# Model-comparison table from 1.b.
print(comparison_1b)
```

```
## # A tibble: 2 × 7
##   model               cv_rmse cv_mse cv_r2 adj_r2   aic   bic
##   <chr>             <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Full model (part a)    10.1  102. 0.717 0.725 3151. 3195.
## 2 Stepwise-selected model (part b) 10.0  100. 0.720 0.726 3147. 3184.
```

Discussion (Question 1d).

1.d **Compare R² and MSE** The R² and MSE values drop slightly in the stepwise-selected model (part b), lower R² and RMSE mean better performance.

e). Using a residuals plot for part a and part b models, state the appropriateness of using a linear model for this problem.

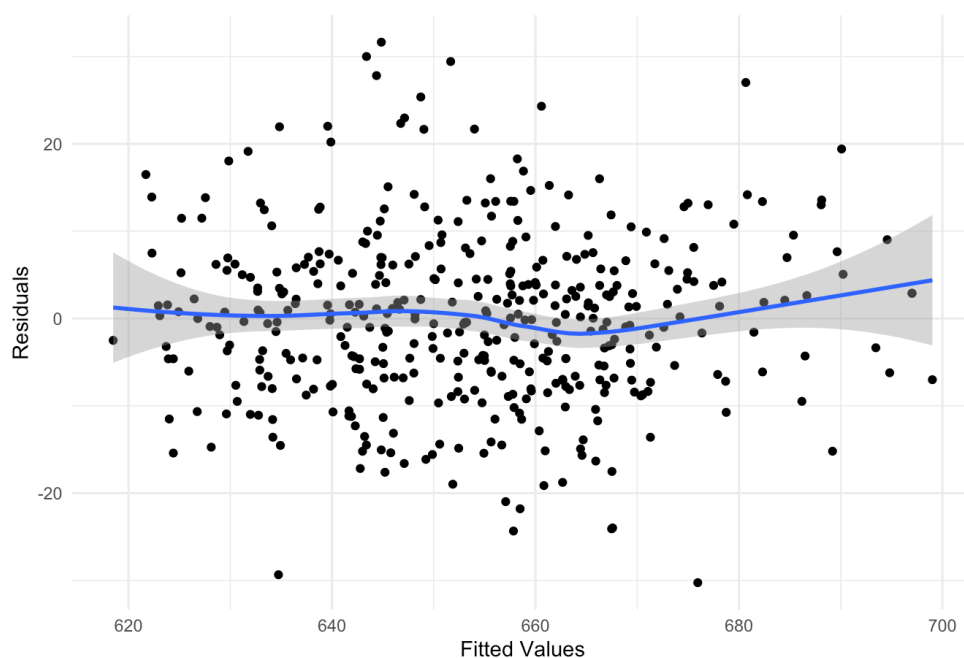
Hide

```
library(ggplot2)

# Create a data frame of fitted values and residuals
full_df <- data.frame(Fitted = fitted(model_full), Residuals = resid(model_full))
step_df <- data.frame(Fitted = fitted(model_step), Residuals = resid(model_step))

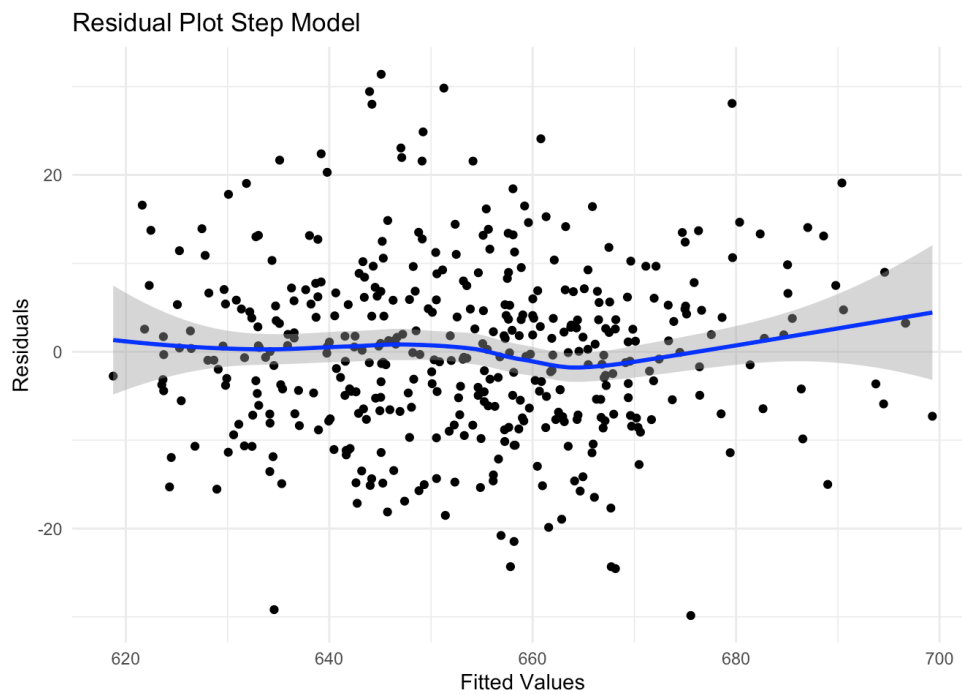
# Generate the full model plot
ggplot(full_df, aes(x = Fitted, y = Residuals)) +
  geom_point() +
  geom_smooth(method = "loess") +
  labs(title = "Residual Plot Full Model", x = "Fitted Values", y = "Residuals") +
  theme_minimal()
```

Residual Plot Full Model



Hide

```
# Generate the step model plot
ggplot(step_df, aes(x = Fitted, y = Residuals)) +
  geom_point() +
  geom_smooth(method = "loess", se = TRUE, color = "blue") + # Add a trend line
  labs(title = "Residual Plot Step Model", x = "Fitted Values", y = "Residuals") +
  theme_minimal()
```

Hide

```
# source: https://rpubs.com/sharmaar3/Residual\_Plots
```

Discussion (Question 1e).

1.e **Why a linear model is appropriate here** Using regression analysis allows us to understand the relationship between the variables, which predictors are statistically significant (and which are not), and provides a confidence level for each coefficient. Finally, regression analysis is easier to understand and explain to others vs a more complicated model (black box).

The following reasons make it appropriate to use a Linear model:

- * Linear relationship: The lunch variable shows clear linear pattern.
- * Continuous Target: The math score is continuous and numerical.
- * Independence: Each observation is independent of the others.

f). Incorporate an interaction term into your multiple linear regression and respond to the following:

1. Provide analytical justification for your interaction term.
2. Compare your results with the models from parts a) and b).

Hide

```
model_f <- lm(math ~ . + lunch:income, data = train)
summary(model_f)
```

```
##
## Call:
## lm(formula = math ~ . + lunch:income, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -28.734  -6.415  -0.337   5.758  31.865
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.611e+02  5.504e+00 120.110 < 2e-16 ***
## gradesKK-08  -4.129e+00  1.608e+00  -2.568  0.01068 *
## students     -1.124e-03  1.815e-03  -0.619  0.53604
## teachers      1.026e-02  4.034e-02   0.254  0.79933
## calworks     -1.022e-01  7.212e-02  -1.417  0.15743
## lunch        -2.883e-01  5.653e-02  -5.100  5.76e-07 ***
## computer      6.155e-03  3.733e-03   1.649  0.10014
## expenditure   9.890e-04  9.946e-04   0.994  0.32082
## income        6.968e-01  1.195e-01   5.829  1.33e-08 ***
## english      -8.032e-02  2.775e-02  -2.894  0.00406 **
## lunch:income  -9.263e-03  3.711e-03  -2.496  0.01306 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 10.01 on 327 degrees of freedom
## Multiple R-squared:  0.7356, Adjusted R-squared:  0.7275
## F-statistic: 90.97 on 10 and 327 DF,  p-value: < 2.2e-16
```

Hide

```
# pred_f <- predict(model_f, newdata = test)
# mse_f <- mean((test$math - pred_f)^2)
summary(fit1)
```

```
##
## Call:
## lm(formula = math ~ ., data = data_1a)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -30.251  -6.671   0.029   5.680  31.649
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  6.622e+02  4.750e+00 139.419 < 2e-16 ***
## gradesKK-08  -3.674e+00  1.409e+00  -2.608  0.00945 **
## students     -6.223e-04  1.727e-03  -0.360  0.71871
## teachers     -1.133e-03  3.804e-02  -0.030  0.97625
## calworks     -1.003e-01  6.622e-02  -1.515  0.13048
## lunch        -3.679e-01  3.719e-02  -9.892 < 2e-16 ***
## computer      5.663e-03  3.272e-03   1.731  0.08422 .
## expenditure   6.430e-04  8.682e-04   0.741  0.45933
## income        6.562e-01  1.015e-01   6.467  2.82e-10 ***
## english      -8.350e-02  2.626e-02  -3.179  0.00159 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.886 on 413 degrees of freedom
## Multiple R-squared:  0.7305, Adjusted R-squared:  0.7247
## F-statistic: 124.4 on 9 and 413 DF,  p-value: < 2.2e-16
```

Hide

Discussion (Question 1f).

f.1 Part a uses the full model including variable predictors for both `lunch`, a measure of poverty or needs, and `income`, a district-level resource. The link between lunch eligibility and math scores can be used to determine whether the interaction is greater than any of the predictors alone.

f.2 In comparison between models from part a) and b) we find that....

g). Fit a 'Ridge Regression' Model for the dataset and interpret the results.

h). Fit a 'LASSO Regression' model for the dataset and interpret the results.

```
library(glmnet)
```

```
## Loading required package: Matrix
```

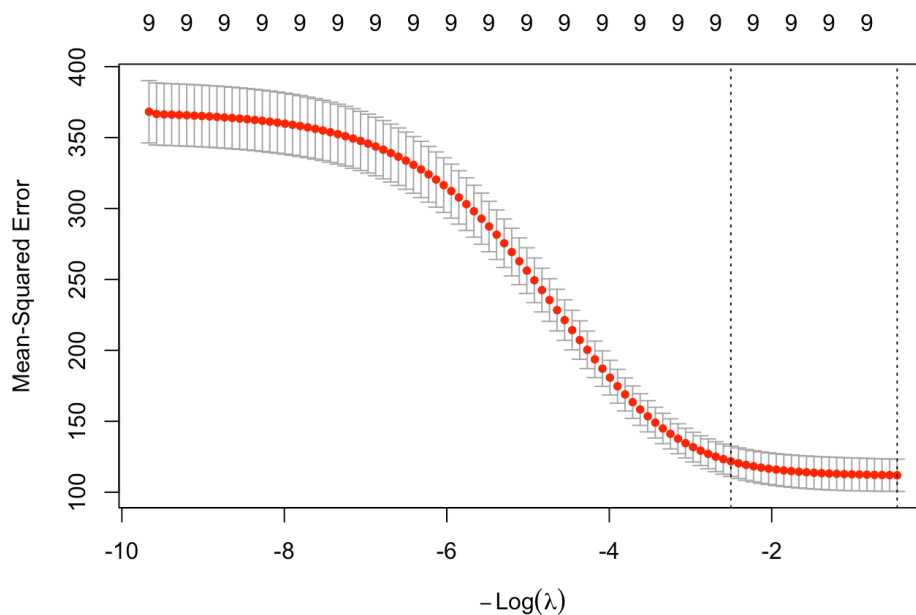
```
##  
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':  
##  
##   expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

Hide

```
x_train <- model.matrix(math ~ ., train)[, -1]  
y_train <- train$math  
x_test  <- model.matrix(math ~ ., test)[, -1]  
y_test  <- test$math  
  
set.seed(335)  
# Ridge (alpha = 0)  
cv_ride <- cv.glmnet(x_train, y_train, alpha = 0)  
plot(cv_ride)
```



Hide

```
pred_ride <- predict(cv_ride, s = cv_ride$lambda.min, newx = x_test)  
mse_ride <- mean((y_test - pred_ride)^2)  
  
# View selected Ridge coefficients at lambda = 4  
cat("View selected Ridge coefficients at lambda = 4\n")
```

```
## View selected Ridge coefficients at lambda = 4
```

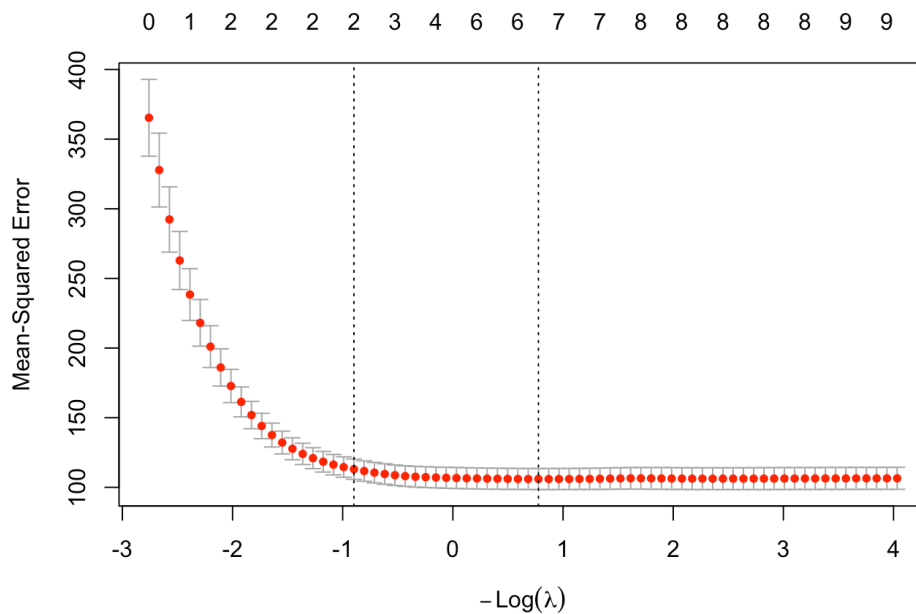
Hide

```
coef(cv_ride, s = 4)
```

```
## 10 x 1 sparse Matrix of class "dgCMatrix"
##               s=4
## (Intercept)  6.559521e+02
## gradesKK-08 -3.461873e+00
## students    -1.909462e-04
## teachers    -3.021931e-03
## calworks    -2.145854e-01
## lunch       -2.721496e-01
## computer     2.580271e-03
## expenditure  1.248793e-03
## income       6.670141e-01
## english     -1.025508e-01
```

Hide

```
# LASSO (alpha = 1)
cv_lasso <- cv.glmnet(x_train, y_train, alpha = 1)
plot(cv_lasso)
```



Hide

```
pred_lasso <- predict(cv_lasso, s = cv_lasso$lambda.min, newx = x_test)
mse_lasso <- mean((y_test - pred_lasso)^2)

# View selected LASSO coefficients
cat("View selected LASSO coefficients\n")
```

```
## View selected LASSO coefficients
```

Hide

```
coef(cv_lasso, s = cv_lasso$lambda.min)
```

```
## 10 x 1 sparse Matrix of class "dgCMatrix"
##              s=0.4599683
## (Intercept)  6.604265e+02
## gradesKK-08 -2.249351e+00
## students      .
## teachers      .
## calworks     -3.891629e-02
## lunch        -4.021128e-01
## computer      .
## expenditure   8.464456e-04
## income        6.204137e-01
## english      -5.679578e-02
```

Discussion (Question 1g and 1h). Ridge regressions advantage over least squares is rooted in the bias-variance trade-off. As λ increases, the flexibility of the ridge regression fit decreases, leading to decreased variance but increased bias.

Lasso uses the same setup as ridge to predict math with all other predictors in the train split.

Lasso vs Ridge:

Using `glmnet` with an `alpha = 1` gives Lasso, and `alpha = 0` gives Ridge.

i). Compare all Testing MSE values that you got from parts a,b,f,g and h. Which model do you recommend?

Hide

```
# Testing MSE from a,b,f,g and h - wich one to recomend
library(glmnet)

# Keep model comparison fair: same train/test split and same response.
y_test = test$math

# (a) Full linear model
pred_a = predict(model_full, newdata = test)
mse_a = mean((y_test - pred_a)^2)

# (b) Stepwise linear model
pred_b = predict(model_step, newdata = test)
mse_b = mean((y_test - pred_b)^2)

# (f) Interaction model
pred_f = predict(model_f, newdata = test)
mse_f = mean((y_test - pred_f)^2)

# (g) Ridge: use CV-selected lambda on training data
x_train = model.matrix(math ~ ., train)[, -1]
x_test = model.matrix(math ~ ., test)[, -1]
y_train = train$math

set.seed(335)
cv_ridge = cv.glmnet(x_train, y_train, alpha = 0)
pred_g = predict(cv_ridge, s = cv_ridge$lambda.min, newx = x_test)
mse_g = mean((y_test - pred_g)^2)

# (h) LASSO: use CV-selected lambda on training data
set.seed(335)
cv_lasso = cv.glmnet(x_train, y_train, alpha = 1)
pred_h = predict(cv_lasso, s = cv_lasso$lambda.min, newx = x_test)
mse_h = mean((y_test - pred_h)^2)

# Collect and rank
model_mse = tibble(
  model = c("a: Full OLS", "b: Stepwise OLS", "f: OLS + lunch:income", "g: Ridge (CV lambda.min)", "h: LASSO (CV lambda.min)"),
  test_mse = c(mse_a, mse_b, mse_f, mse_g, mse_h)
) |>
  dplyr::arrange(test_mse) |>
  dplyr::mutate(delta_from_best = test_mse - min(test_mse))

print(model_mse)
```

```
## # A tibble: 5 × 3
##   model                test_mse delta_from_best
##   <chr>                <dbl>         <dbl>
## 1 b: Stepwise OLS        80.6           0
## 2 a: Full OLS           81.3         0.771
## 3 g: Ridge (CV lambda.min) 81.7         1.13
## 4 h: LASSO (CV lambda.min) 82.9         2.35
## 5 f: OLS + lunch:income  83.9         3.36
```

Hide

```
best_model = model_mse$model[1]
cat("\nRecommended by lowest test MSE:", best_model, "\n")
```

```
##
## Recommended by lowest test MSE: b: Stepwise OLS
```

Hide

```
# Optional: if models are very close, choose the simpler one (parsimony).
cat("If top models differ by < 1-2 MSE points, prefer the simpler/interpretable model.\n")
```

```
## If top models differ by < 1-2 MSE points, prefer the simpler/interpretable model.
```

Discussion (Question 1i). I compared models a, b, f, g, and h using the same train/test split. The stepwise model (b) has the lowest test MSE, so this is the best predictive model of the group. Full OLS (part a) and ridge (part g) were close but not better. The stepwise model is a good balance of accuracy and interpretability.

Problem 2: Tree based models:

This question should be answered using the **“Wine Quality”** data set. Description about the data set can be found on the link provided. Objective of this question is to fit an regression tree model to predict quality of wine.

- Produce some numerical and graphical summaries of the data set. Explain the relationships.
- Create a training set with 80% of the observations, and a testing set containing the remaining 20%.
- Fit a regression tree with *quality* as the response variable using the training set. Plot the tree and interpret the results. What test MSE do you obtain?
- Use cross-validation in order to determine the optimal level of tree complexity. Does pruning the tree improve the test MSE?
- Use random forests to analyze this data. What test MSE do you obtain?
- Use the *importance()* function to determine which variables are most important.

Hide

```
library(tree)
library(randomForest)
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'
```

```
## The following object is masked from 'package:psych':
##
##   outlier
```

```
## The following object is masked from 'package:dplyr':
##
##   combine
```

```
## The following object is masked from 'package:ggplot2':
##
##   margin
```

Hide

```
# This dataset is semicolon-separated (CSV2-style), not comma-separated.
wine <- read.csv("winequality.csv", sep = ";")
glimpse(wine)
```

```
## Rows: 4,898
## Columns: 12
## $ fixed.acidity      <dbl> 7.0, 6.3, 8.1, 7.2, 7.2, 8.1, 6.2, 7.0, 6.3, 8.1,...
## $ volatile.acidity  <dbl> 0.27, 0.30, 0.28, 0.23, 0.23, 0.28, 0.32, 0.27, 0...
## $ citric.acid        <dbl> 0.36, 0.34, 0.40, 0.32, 0.32, 0.40, 0.16, 0.36, 0...
## $ residual.sugar    <dbl> 20.70, 1.60, 6.90, 8.50, 8.50, 6.90, 7.00, 20.70,...
## $ chlorides          <dbl> 0.045, 0.049, 0.050, 0.058, 0.058, 0.050, 0.045, ...
## $ free.sulfur.dioxide <dbl> 45, 14, 30, 47, 47, 30, 30, 45, 14, 28, 11, 17, 1...
## $ total.sulfur.dioxide <dbl> 170, 132, 97, 186, 186, 97, 136, 170, 132, 129, 6...
## $ density            <dbl> 1.0010, 0.9940, 0.9951, 0.9956, 0.9956, 0.9951, 0...
## $ pH                <dbl> 3.00, 3.30, 3.26, 3.19, 3.19, 3.26, 3.18, 3.00, 3...
## $ sulphates          <dbl> 0.45, 0.49, 0.44, 0.40, 0.40, 0.44, 0.47, 0.45, 0...
## $ alcohol            <dbl> 8.8, 9.5, 10.1, 9.9, 9.9, 10.1, 9.6, 8.8, 9.5, 11...
## $ quality            <int> 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 5, 5, 5, 7, 5, 7, 6...
```

Hide

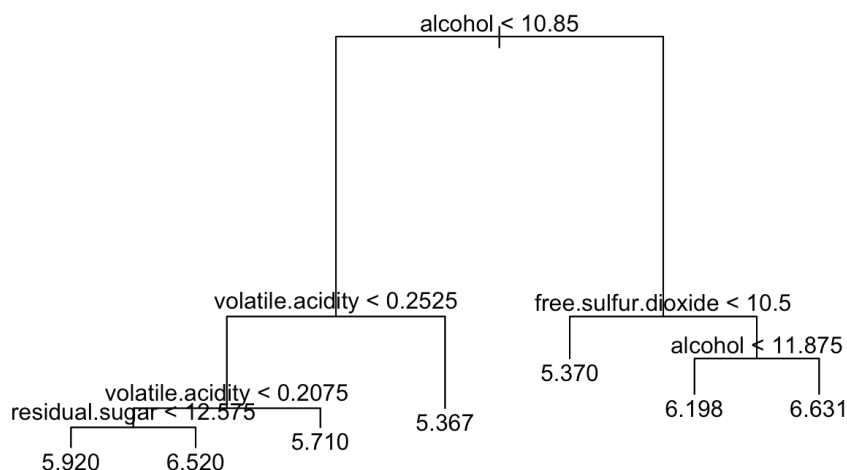
```
summary(wine)
```

```
## fixed.acidity    volatile.acidity    citric.acid      residual.sugar
## Min.   : 3.800    Min.   :0.0800    Min.   :0.0000    Min.   : 0.600
## 1st Qu.: 6.300    1st Qu.:0.2100    1st Qu.:0.2700    1st Qu.: 1.700
## Median : 6.800    Median :0.2600    Median :0.3200    Median : 5.200
## Mean   : 6.855    Mean   :0.2782    Mean   :0.3342    Mean   : 6.391
## 3rd Qu.: 7.300    3rd Qu.:0.3200    3rd Qu.:0.3900    3rd Qu.: 9.900
## Max.   :14.200    Max.   :1.1000    Max.   :1.6600    Max.   :65.800
## chlorides        free.sulfur.dioxide    total.sulfur.dioxide    density
## Min.   :0.00900    Min.   : 2.00      Min.   : 9.0      Min.   :0.9871
## 1st Qu.:0.03600    1st Qu.: 23.00     1st Qu.:108.0     1st Qu.:0.9917
## Median :0.04300    Median : 34.00     Median :134.0     Median :0.9937
## Mean   :0.04577    Mean   : 35.31     Mean   :138.4     Mean   :0.9940
## 3rd Qu.:0.05000    3rd Qu.: 46.00     3rd Qu.:167.0     3rd Qu.:0.9961
## Max.   :0.34600    Max.   :289.00     Max.   :440.0     Max.   :1.0390
## pH              sulphates          alcohol          quality
## Min.   :2.720    Min.   :0.2200    Min.   : 8.00     Min.   :3.000
## 1st Qu.:3.090    1st Qu.:0.4100    1st Qu.: 9.50     1st Qu.:5.000
## Median :3.180    Median :0.4700    Median :10.40     Median :6.000
## Mean   :3.188    Mean   :0.4898    Mean   :10.51     Mean   :5.878
## 3rd Qu.:3.280    3rd Qu.:0.5500    3rd Qu.:11.40     3rd Qu.:6.000
## Max.   :3.820    Max.   :1.0800    Max.   :14.20     Max.   :9.000
```

Hide

```
# Train/test split
set.seed(335)
n <- nrow(wine)
train_idx <- sample.int(n, size = floor(0.8 * n))
train_w <- wine[train_idx, ]
test_w <- wine[-train_idx, ]

# Regression Tree
tree_model <- tree(quality ~ ., data = train_w)
plot(tree_model); text(tree_model, pretty = 0)
```



Hide

```

pred_tree <- predict(tree_model, newdata = test_w)
mse_tree <- mean((test_w$quality - pred_tree)^2)

# Cross-Validation for Pruning
cv_tree <- cv.tree(tree_model, FUN = prune.tree)
best_size <- cv_tree$size[which.min(cv_tree$dev)]
pruned <- prune.tree(tree_model, best = best_size)
pred_pruned <- predict(pruned, newdata = test_w)
mse_pruned <- mean((test_w$quality - pred_pruned)^2)

# Random Forest
rf_model <- randomForest(quality ~ ., data = train_w, mtry = 4, ntree = 500, importance = TRUE)
pred_rf <- predict(rf_model, newdata = test_w)
mse_rf <- mean((test_w$quality - pred_rf)^2)

# Variable Importance
importance(rf_model)

```

```

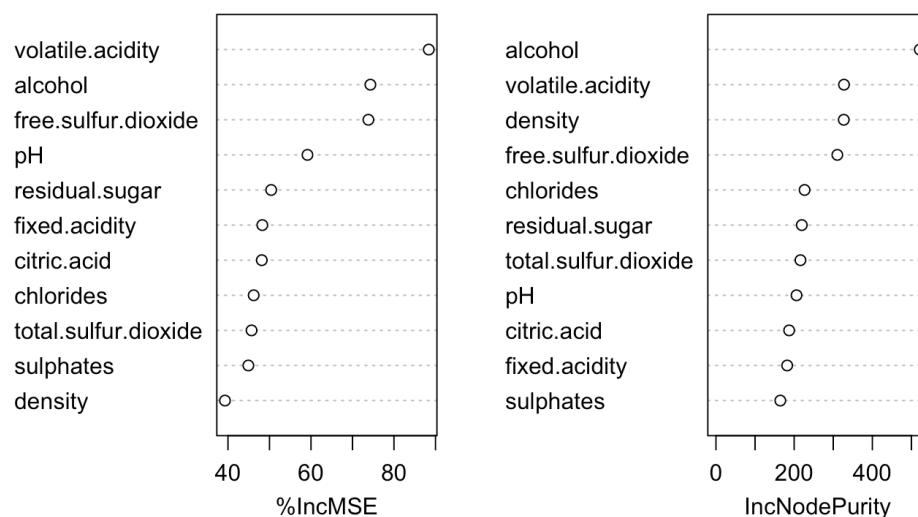
##          %IncMSE IncNodePurity
## fixed.acidity      48.27332      182.1423
## volatile.acidity    88.38633      327.4816
## citric.acid         48.14570      187.2059
## residual.sugar      50.40807      219.5846
## chlorides           46.19339      226.7109
## free.sulfur.dioxide  73.83366      310.4862
## total.sulfur.dioxide 45.67763      216.1741
## density             39.27050      326.9372
## pH                  59.15420      206.0919
## sulphates           44.90582      164.6352
## alcohol             74.31511      521.3995

```

Hide

```
varImpPlot(rf_model)
```


rf_model



Hide

```
# Display results
wine_model_mse = tibble(
  model = c("Tree", "Pruned", "RF"),
  test_mse = c(mse_tree, mse_pruned, mse_rf)
) |>
  dplyr::arrange(test_mse) |>
  dplyr::mutate(delta_from_best = test_mse - min(test_mse))

print(wine_model_mse)
```

```
## # A tibble: 3 × 3
##   model test_mse delta_from_best
##   <chr>   <dbl>         <dbl>
## 1 RF      0.424             0
## 2 Tree    0.623           0.199
## 3 Pruned  0.623           0.199
```

Question 2 Discussion The `print(wine_model_mse)` shows that Random Forest has the smallest MSE 0.424 while both the full tree and pruned tree are about the same value of 0.623. This means that pruning did not help and Random Forest performed the best. Cross validation selected the pruned tree but pruning did not improve test performance (test MSE stayed about the same 0.623). The Random Forest achieved the lowest test MSE, substantially better than both tree models, so it is the preferred model for prediction on this data set.

The variable importance results indicate that alcohol is the strongest predictor of wine quality, with secondary contributions from variables such as volatile.acidity, density and free.sulfur.dioxide while remaining predictors have comparatively smaller effects.